

Motion Controller Programming Reference Manual

Table of Contents

Motion Controller Programming Reference Manual	1
1. Introduction	1
2. Device Boot-up and Communication States	1
2.1 Pre-operational State	1
2.2 Example – Accessing Parameters in Pre-operational State	1
Read Device Type	1
Set Heartbeat Time	2
2.3 Transition to Operational State	2
3. Device Profile Specific Parameters (CiA 402)	2
3.1 Controlword (0x6040)	2
3.2 Statusword (0x6041)	3
3.3 Halt Option Code (0x605D)	4
3.4 Modes of Operation (0x6060)	4
3.5 Modes of Operation Display (0x6061)	4
3.6 Position Demand Value (0x6062)	5
3.7 Position Measured Value (0x6064)	5
3.8 Velocity Demand Value (0x606B)	5
3.9 Velocity Measured Value (0x606C)	5
3.10 Target Position (0x607A)	6
3.11 Target Acceleration (0x6083)	6
3.12 Target Deceleration (0x6084)	6
3.13 Target Velocity (0x60FF)	6
4. RPDO Mappings and Parameter Access	7
4.1 RPDO1 – Control, Mode, and Torque Command	7
4.2 RPDO2 – Current Loop Parameters	7
4.3 RPDO3 – Speed Loop Parameters	7
4.4 RPDO4 – Position Loop Parameters	8
4.5 RPDO5 – Motion Target Commands	8

4.6 RPDO6 – Acceleration/Deceleration.....	8
5. TPDO Mappings and Monitoring.....	9
5.1 TPDO1 – Position Feedback.....	9
5.2 TPDO2 – Velocity Feedback.....	9
5.3 TPDO3 – Current Feedback	9
5.4 TPDO4 – Commanded Motion Values.....	10
6. Example Communication Flow	10
7. Summary	10

1. Introduction

The Motion Controller is a CANopen-compliant servo/motor controller implementing **CiA 301 (CANopen communication profile)** and **CiA 402 (Drives and Motion Control profile)**. It supports control via both **SDO (Service Data Object)** and **PDO (Process Data Object)** communication mechanisms.

After power-on, the device initializes and enters the **Pre-operational state**. In this state, only SDO communication is available, allowing configuration of parameters before enabling runtime data exchange.

2. Device Boot-up and Communication States

2.1 Pre-operational State

After the device powers on, it enters the **Pre-operational state** automatically.

- Only **SDO communication** is active in this state.
- **PDOs (RPDOs and TPDOs)** are **not active** yet.
- The user can configure all communication and motion parameters through SDO writes.
- The controller does **not execute motion** commands in this state.

This allows safe configuration before enabling motion or control loops.

2.2 Example – Accessing Parameters in Pre-operational State

Read Device Type

To verify communication, the master can read the device type located at **index 0x1000**.

Field	Value
Index	0x1000
Sub-index	0x00
Type	UNSIGNED32
Example Value	0x00000001

Example SDO Read (CANopen frame)

COB-ID: 0x600 + NodeID
Command: 0x40 (read request)
Data: 00 10 00 00 00 00 00 00

Response:

COB-ID: 0x580 + NodeID
Data: 43 00 10 00 01 00 00 00

Set Heartbeat Time

To configure the heartbeat period (object 0x1017), write a 16-bit value in milliseconds.

Field	Value
Index	0x1017
Sub-index	0x00
Type	UNSIGNED16
Example	1000 (1 second)

Example SDO Write (1s Heartbeat)

COB-ID: 0x600 + NodeID
Data: 2B 17 10 00 E8 03 00 00

2.3 Transition to Operational State

After configuration, send an NMT command to switch to **Operational state**.

Command	Hex Code	Description
Start Remote Node	0x01	Enables PDO communication

Example CAN frame:

COB-ID: 0x000
Data: 01 <NodeID>

Once in the operational state: - All **RPDOs** and **TPDOs** become active. - Controlwords and targets can be written via PDOs. - The motion controller executes the commanded profile.

3. Device Profile Specific Parameters (CiA 402)

This section describes all CiA 402 standard objects supported by the Motion Controller. These objects define how the drive is controlled, commanded, and monitored.

3.1 Controlword (0x6040)

Property	Value
Data Type	UNSIGNED16
Access	rw / PDO-mapped
Default	0

The **Controlword** is used to command transitions in the CiA 402 state machine.

Example Sequence:

1. Write 0x0006 → Switch on.
2. Write 0x0007 → Enable voltage.
3. Write 0x000F → Enable operation.

In the current implementation, you can directly send **15 (0x000F)** which will immediately activate the drive and enable motion.

3.2 Statusword (0x6041)

Property	Value
Data Type	UNSIGNED16
Access	ro
Default	0

The **Statusword** reports the current state of the drive and motion execution.

Typical bits:

Bit	Meaning
0	Ready to switch on
1	Switched on
2	Operation enabled
3	Fault present
4	Voltage enabled
5	Quick stop active
6	Switch on disabled
10	Target reached
12	Following error

This object is read-only and used by the master to monitor drive readiness.

3.3 Halt Option Code (0x605D)

Property	Value
Data Type	UNSIGNED8
Access	rw
Default	0

Defines the behavior when the **Halt bit** (bit 8 in the Controlword) is set.

Value	Description
0	Immediate stop (freewheel or quick stop)
1	Ramp stop using deceleration (0x6084)
2	Ramp stop and allow resume when Halt clears

3.4 Modes of Operation (0x6060)

Property	Value
Data Type	INTEGER8
Access	rw
Default	1

Defines the active motion mode.

Value	Mode
1	Profile Position Mode
3	Profile Velocity Mode
4	Torque (Iq) Control Mode

3.5 Modes of Operation Display (0x6061)

Property	Value
Data Type	INTEGER8
Access	ro
Default	0

Displays the **currently active mode**. This may differ temporarily from the commanded mode if the change is pending.

3.6 Position Demand Value (0x6062)

Property	Value
Data Type	REAL32
Access	ro / TPDO-mapped
Default	0

Represents the interpolated position command that the drive follows. Used for monitoring commanded trajectories.

3.7 Position Measured Value (0x6064)

Property	Value
Data Type	REAL64
Access	rw / TPDO-mapped
Default	0

The **measured position feedback** from the encoder or resolver. Unit: revolutions.

3.8 Velocity Demand Value (0x606B)

Property	Value
Data Type	REAL32
Access	ro / TPDO-mapped
Default	0

Represents the instantaneous velocity that the profile generator requests from the drive.

3.9 Velocity Measured Value (0x606C)

Property	Value
Data Type	REAL32
Access	ro / TPDO-mapped

Property	Value
Default	0

The **actual measured velocity** from feedback (rps). Useful for control loop validation.

3.10 Target Position (0x607A)

Property	Value
Data Type	REAL32
Access	rw / RPDO-mapped
Default	0

Specifies the desired target position in **revolutions**. Used in **Profile Position Mode**.

3.11 Target Acceleration (0x6083)

Property	Value
Data Type	UNSIGNED32
Access	rw / RPDO-mapped
Default	1000

Defines the target acceleration rate in **rps²**. Multiply by 100 before sending the value.

3.12 Target Deceleration (0x6084)

Property	Value
Data Type	UNSIGNED32
Access	rw / RPDO-mapped
Default	1000

Defines the target deceleration rate in **rps²**. Multiply by 100 before sending the value.

3.13 Target Velocity (0x60FF)

Property	Value
Data Type	REAL32

Property	Value
Access	rw / RPDO-mapped
Default	0

Specifies the **desired velocity command** in **revolutions per second**. Used in **Profile Velocity Mode**.

4. RPDO Mappings and Parameter Access

4.1 RPDO1 – Control, Mode, and Torque Command

COB-ID: \$NODEID + 0x200

Mapping (0x1600):

- 0x6040: Controlword (U16)
- 0x6060: modes_of_operation (I8)
- 0x0002: Dummy0002 (I8) — reserved filler byte, ignore
- 0x2030: target_iq (REAL32)

This PDO is intended for quickly placing the drive into the desired mode and issuing a torque (Iq) command. The filler byte keeps the PDO length aligned.

4.2 RPDO2 – Current Loop Parameters

Object	Name	Description
0x2200	iq_kp	Proportional gain for q-axis current loop.
0x2201	iq_ki	Integral gain for q-axis current loop.

These parameters tune the inner current controller. They can be sent together in one PDO frame mapped to RPDO2.

Example:

COB-ID: NodeID + 0x300

Data: <iq_kp_4B> <iq_ki_4B>

4.3 RPDO3 – Speed Loop Parameters

Object	Name	Description
0x2202	speed_kp (*)	Proportional gain for speed control.

Object	Name	Description
0x2203	speed_ki (*)	Integral gain for speed control.
0x2204	speed_kd (*)	Derivative gain for speed control.
0x2205	speed_Tc (*)	Derivative filter time constant.

This RPDO configures the outer speed loop. The parameters are written simultaneously via a single PDO message.

Mapping

COB-ID: \$NodeID + 0x400

POS - 0 2 4 6 7

Data: <speed_kp_2B> <speed_ki_2B> <speed_kd_2B> <speed_Tc_2B>

Note - * says multiply by 100 before sending your value. For eg – if you want to set Kp as 0.3, send $0.3 \times 100 = 30$.

4.4 RPDO4 – Position Loop Parameters

Object	Name	Description
0x2206	pos_kp (*)	Proportional gain for position loop.
0x2207	pos_ki (*)	Integral gain for position loop.
0x2208	pos_kd (*)	Derivative gain for position loop.
0x2209	pos_Tc (*)	Derivative filter constant.

Used to configure the outermost control loop for precise position tracking.

Mapping

COB-ID: \$NodeID + 0x500

POS - 0 2 4 6 7

Data: <Pos_kp_2B> <Pos_ki_2B> <Pos_kd_2B> <Pos_Tc_2B>

Note - * says multiply by 100 before sending your value. For eg – if you want to set Kp as 0.3, send $0.3 \times 100 = 30$.

4.5 RPDO5 – Motion Target Commands

Object	Name	Description
0x607A	target_position	Desired target position (revolutions).
0x60FF	target_velocity	Desired velocity (revolutions per second).

COB-ID: 0x680 + \$NodeID

Mapping (0x1604): 0x607A (32-bit, float), 0x60FF (32-bit, float)

This PDO allows sending motion commands in operational mode. The controller uses the configured acceleration and deceleration ramps for profile generation.

4.6 RPDO6 – Acceleration/Deceleration

Object	Name	Description
0x6083	target_acceleration	Profile acceleration (rps ²). Multiply by 100 before sending.
0x6084	target_deceleration	Profile deceleration (rps ²). Multiply by 100 before sending.

COB-ID: 0x690 + \$NodeID

Mapping (0x1605): 0x6083 (uint32_t), 0x6084 (uint32_t)

Use this PDO to update ramp rates during operation.

This PDO allows sending motion commands in operational mode. The controller uses the configured acceleration and deceleration ramps for profile generation.

5. TPDO Mappings and Monitoring

5.1 TPDO1 – Position Feedback

Object	Name	Description
0x6064	position_measured	Actual position feedback from the encoder, in revolutions.

COB-ID: \$NODEID + 0x180

Mapping (0x1A00): 0x6064 (REAL64)

Used to monitor the real-time position feedback of the motor.

5.2 TPDO2 – Velocity Feedback

Object	Name	Description
0x606C	velocity_measured	Actual velocity feedback in revolutions per second.

COB-ID: \$NODEID + 0x280

Mapping (0x1A01): 0x606C (REAL32)

Provides periodic velocity feedback, useful for speed loop tuning or verification.

5.3 TPDO3 – Current Feedback

Object	Name	Description
0x2032	iq_measured	q-axis current feedback in amperes.
0x2033	id_measured	d-axis current feedback in amperes.

COB-ID: \$NODEID + 0x380

Mapping (0x1A02): 0x2032 (REAL32), 0x2033 (REAL32)

This TPDO transmits motor current components for monitoring or diagnostics.

5.4 TPDO4 – Commanded Motion Values

Object	Name	Description
0x6062	position_demand_value	Commanded position after profile generation.
0x606B	velocity_demand_value	Commanded velocity after profile generation.

COB-ID: \$NODEID + 0x480

Mapping (0x1A03): 0x6062 (REAL32), 0x606B (REAL32)

Provides insight into what the internal motion planner is commanding in real time.

6. Example Communication Flow

1. Power on → Device enters Pre-operational.

2. Configure heartbeat and loop gains using SDOs.
 3. Send NMT command to enter Operational state.
 4. Start sending RPDOs to command motion.
 5. Read feedbacks (position, velocity, current) via TPDOs.
-

7. Summary

This programming reference provides the fundamental steps and object mappings required to control the Motion Controller over CANopen. Using the described SDO and PDO interfaces, the user can configure, tune, and command the device through all layers of motion control hierarchy — from torque to position control.